

Game-Theoretic-GAI Approach for Computation Offloading and Resource Management for Mobile Edge Collaborative Vehicular Networks

Nusrat Jahan, Mohammad Kamrul Hasan[✉], *Senior Member, IEEE*, Shayla Islam[✉], *Senior Member, IEEE*,
Mohd Zakree Ahmad Nazri[✉], Khairul Akram Zainol Ariffin, Huda Saleh Abbas,
Ali Alqahtani[✉], and Hardik Gohel

Abstract—To counter challenges posed by emerging service needs, limited resources, and the imperative of real-time flexibility, the present research synthesizes game theory and Generative Artificial Intelligence (GAI). The designed architecture supports discreet and scalable resource allocation by combining a Stackelberg game framework with GAI-driven simulation and decision-support mechanisms. The algorithmic methodology of two stages allocates resources among vehicles and Mobile Edge Computing (MEC) servers in an efficient manner, optimizing the offloading ratio. The architecture dynamically adapts by changing network environments using GAI's ability to simulate complex vehicular scenarios and weight trade-offs concerning latency, energy consumption, and computation efficiency. The resource allocation process is supported by an intelligent offloading decision-support module powered by GAI, complementing parameter optimization using Lagrangian optimization and Karush-Kuhn-Tucker (KKT) conditions. The performance of the designed framework is validated using extensive simulations. The results show the reductions in computational overhead, energy consumption, and latency compared to conventional methods, especially in cases of dynamic job intensity and communication scenarios. The Generative AI can improve the system scalability by allowing task offloading in resource-scarce cases. The result demonstrates the synergistic effectiveness of GAI and game-theoretical approaches in resolving real-time resource allocation challenges in vehicle-to-everything networks.

Index Terms—Game-theoretic frameworks, generative AI, computation offloading, mobile edge computing (MEC), vehicular networks, next-generation ITS applications.

Received 13 January 2025; revised 20 March 2025 and 27 May 2025; accepted 28 May 2025. This work was conducted and supported by the National University of Malaysia under the Research Grant Scheme No. GUP 2023-010. The Associate Editor for this article was T. R. Gadekallu. (Corresponding authors: Mohammad Kamrul Hasan; Shayla Islam; Nusrat Jahan.)

Nusrat Jahan, Mohammad Kamrul Hasan, Mohd Zakree Ahmad Nazri, and Khairul Akram Zainol Ariffin are with the Center for Cyber Security, Faculty of Information Science and Technology (FTSM), Universiti Kebangsaan Malaysia (UKM), Selangor 43600, Malaysia (e-mail: p128645@siswa.ukm.edu.my; mkhasan@ukm.edu.my).

Shayla Islam is with the Institute of Computer Science and Digital Innovation, UCSI University, Kuala Lumpur 56000, Malaysia (e-mail: shayla@ucsiuniversity.edu.my).

Huda Saleh Abbas is with the Computer Science Department, Computer Science and Engineering College, Taibah University, Madinah 42353, Saudi Arabia (e-mail: Habbas@taibahu.edu.sa).

Ali Alqahtani is with the Department of Networks and Communications Engineering, College of Computer Science and Information Systems, Najran University, Najran 61441, Saudi Arabia (e-mail: asalqahtany@nu.edu.sa).

Hardik Gohel is with the Applied AI Research Laboratory, Texas A&M University–Victoria, Victoria, TX 77901 USA (e-mail: gohelh@tamu.edu).

Digital Object Identifier 10.1109/TITS.2025.3577673

I. INTRODUCTION

THE integration of edge computing into vehicular networks represents a significant advancement in improving Quality of Service (QoS) by enabling computational tasks to be offloaded from vehicles to nearby edge servers [1]. Unlike traditional cloud computing, edge computing reduces latency and communication overhead by placing resources closer to data sources [2]. This proximity enhances efficiency, making it well-suited for intelligent transportation systems (ITS) applications. Still, a major difficulty stays in properly controlling compute offloading and resource allocation given the dynamic character of service requests and resource availability in these networks.

Greedy algorithms, convex optimization techniques, and game-theoretic frameworks are among current approaches to handle compute offloading issues. Although these methods have been encouraging, they usually treat offloading choices and resource allocation separately, hence missing any synergies. Game-theoretic models like Stackelberg and Nash Equilibrium frameworks have efficiently combined these elements [3]. For example, Stackelberg games have balanced competing goals like latency reduction and energy efficiency in mobile edge computing (MEC) contexts [4]. Despite their promise, these techniques struggle with implementation complexity, data accuracy, and real-world adaptation.

By producing synthetic yet realistic data and improving predictive models, the GAI has become a revolutionary answer to these problems, enabling dynamic and informed decision-making in computation offloading and resource management and simulating complex scenarios and optimizing decision-making processes [5]. Its inclusion into vehicle networks allows for real-time adjustment. It creates road design for cities to prevent congestion [6], hence enabling edge servers and cars to maximize resource allocation and task distribution under different network conditions. GAI has been tested for enhancing vehicular network performance through traffic pattern simulation [7], adaptive communication protocol generation, and cybersecurity improvements [8]. These applications enhance network scalability and resilience, supporting the optimization of communication and computation processes. For example, generative diffusion models have been integrated with reinforcement learning strategies to improve task offloading and resource allocation in MEC systems [9].

Additionally, federated learning techniques combined with Generative AI have addressed privacy concerns by enabling decentralized training and inference, reducing dependence on centralized cloud infrastructure.

In traffic management, GAI models, particularly Generative Adversarial Networks (GANs), were employed to simulate diverse traffic conditions using historical data collected from sensors and cameras across metropolitan areas. This approach enabled dynamic routing recommendations and optimized signal timings, resulting in a 20% reduction in traffic congestion during peak hours and a 15% decrease in carbon emissions due to reduced idle times [10]. GAI methods—including Variational Autoencoders (VAEs) and GANs—were applied in vehicle communication security to provide synthetic attack scenarios simulating a broad spectrum of cyber risks. With the network effectively identifying and responding to 90% of simulated attacks, these models greatly improved the defensive capacity of vehicle communication systems [11]. These developments show how well GAI handles security and operational issues in vehicle networks, hence opening the path for more sustainable, secure, and efficient smart transportation systems.

A. Motivation

The exponential rise of Intelligent Transportation Systems (ITS) and autonomous vehicles has greatly raised computational needs, requiring sophisticated resource management and offloading solutions in MEC-supported vehicular networks [12]. Although MEC enables localised processing and hence lowers latency and energy use, vehicle environments are affected by job heterogeneity, changing network circumstances, and resource constraints. Recent developments in Generative Artificial Intelligence (GAI) have brought models, including the Generative Diffusion Model with TD3 for MEC in Wi-Fi networks [9], GAN-based frameworks for traffic simulation and communication optimisation [10], and federated GAI-IoV architectures for edge intelligence [13]. Though they improve traffic-aware computation and forecasting ability, these methods sometimes ignore multi-agent coordination and strategic resource negotiation. Furthermore, large-scale GAI models create edge deployment problems because of memory limits and privacy issues, problems somewhat lessened by model compression and distributed inference methods. This paper offers a new GAI-driven two-stage compute offloading system, carefully optimised with a Stackelberg game-theoretic model to fill in these shortcomings. The system sets a benchmark for next-generation ITS applications by combining GAI-based task simulation with hierarchical leader-follower optimisation and KKT-based tuning, therefore supporting scalable, adaptive, and energy-efficient decision-making under different vehicular and network conditions.

B. Contributions

This study's major findings are as follows:

- This work presents the framework combining Generative Artificial Intelligence (GAI) with the Stackelberg game model to address dynamic service demands,

resource constraints, and real-time adaptability in vehicular networks, therefore enabling strategic resource allocation and enhancing scalability and robustness.

- This study analyses a two-stage computation model that optimizes offloading ratios and resource allocation between vehicles and MEC servers, balancing latency, energy consumption, and computational efficiency through Lagrangian optimization and KKT conditions to dynamically adapt to varying network conditions for efficient handling of computationally intensive and delay-sensitive tasks in vehicular environments.
- Study the simulation evaluation of the two-stage computation model's effectiveness, in terms of computational overhead, energy consumption, and latency in next-generation intelligent transportation systems.

This paper comprises six sections: Section II reviews related work on vehicular computation offloading; Section III outlines the basic offloading structure; Section IV details the system model and problem formulation for two-stage offloading; Section V presents results, performance comparisons, and Generative AI-based analysis; and Section VI concludes with a summary of findings. A comparative table and algorithmic solutions are also included to highlight contributions and evaluate effectiveness.

II. BACKGROUND

Edge computing integration into vehicular networks aims to improve Quality of Service (QoS) by transferring computational tasks from vehicles to nearby edge servers [14]. Unlike traditional cloud computing, which suffers from increased communication overhead and delays in data transmission [15], edge computing alleviates these issues by placing computational resources closer to where data is generated [16]. However, effectively managing computation offloading and resource management remains challenging because of the behavior of service requests and resource availability.

Several studies have addressed these challenges through edge-cloud computing offloading decision (OD) and resource allocation (RA) strategies. For instance, [17] Chen et al. proposed a greedy algorithm to minimize application finishing time, while Ren et al. formulated RA as a convex optimization problem. Despite these efforts, existing approaches often treat OD and RA independently, missing potential synergies from a combined approach.

Research on computation offloading in vehicular networks addresses rising computational demands through game-theoretic approaches and advanced algorithms. Non-cooperative game theory optimizes UAV offloading but faces implementation challenges. The Stackelberg game model enhances cloud-edge utilities but needs empirical validation. Game theory improves 5G offloading via D2D and MEC but requires dynamic network validation. A two-stage model integrates game theory and Q-learning for resource allocation, showing promise but needing broader testing. Though they have complexity and data accuracy concerns, other approaches, such as contract theory and reinforcement learning, provide creative answers.

This study [9] suggested a paradigm for MEC in Wi-Fi networks combining Twin Delayed DDPG (TD3) with the Generative Diffusion Model (GDM). Using reinforcement learning and the Hungarian algorithm for resource allocation, the method tackles the sparse sample problem and lowers latency and energy use. In this manner, [18] this study explores the integration of Generative AI (GAI) with game-theoretic models, particularly the Stackelberg game, to enhance computation offloading and resource allocation in 6G-enabled vehicular networks. By combining predictive modeling with strategic decision-making, the approach improves energy efficiency, latency reduction, and edge intelligence, despite challenges like scalability and privacy.

On the other hand, this study [19] aims to lower latency in metaverse applications by dynamically splitting and offloading AI-generated content (AIGC) workloads among several edge servers. The study work [20], on the other hand, uses Generative AI and reinforcement learning for load balancing across roadside units (RSUs) and vehicle trajectory prediction. Although all three methods use artificial intelligence to optimise edge computing, the game-theoretic model stresses incentive-driven resource management, while the other two give adaptive offloading top priority—one for multimodal AIGC services and the other for dynamic RSU task balancing in vehicle networks.

Although some offloading techniques help to develop the subject, more validation is required to determine real-world scalability. The research presents an overview of diverse methods for optimizing computation offloading in vehicular networks, each with strengths and specific challenges in Table I. These approaches collectively advance the field, yet further validation and refinement are essential for real-world robustness and scalability.

III. COMPUTATION OFFLOADING

A. Process of Executing Computation Offloading

The execution of computation offloading for users, particularly for computation-intensive tasks, involves a series of five steps:

Task Division: The computation-intensive tasks are first divided into multiple subtasks based on the specific nature of each task. During this process, it is crucial to ensure that the functional integrity of each subtask is preserved.

Offloading Decision-Making: In this step, a decision is made regarding whether the computation-intensive tasks should be offloaded. Determining the best way and place to offload is also part of this choice.

Task Deployment: The subtasks are distributed to the assigned edge servers with the required resources to manage them once the decision to offload has been made.

Task Processing: The edge servers run the subtasks in this phase.

Results Returning: The edge servers send the outcomes back to the users following completion of subtask processing. Figure 1 shows the whole procedure for running user computation offloading.

B. Partial Offloading

Partial offloading enables splitting computation tasks between local devices and edge servers as shown in Figure 2, offering flexibility but increased complexity due to cooperative execution and proportion decision-making. Influenced by user preferences, network conditions, and device capabilities, recent research has explored optimization strategies in MEC environments. Studies have examined energy-harvesting [28] systems, multi-hop task offloading [29], and incentive-based frameworks [30]. Compared to full offloading, partial offloading aims to minimize delay and energy consumption, while maximizing system utility and operational efficiency. The Stackelberg game model enables real-time adaptability by structuring decision-making into a leader-follower hierarchy, where the MEC server allocates resources, and vehicles optimize offloading decisions accordingly. The integration of Generative AI enhances this adaptability by providing predictive insights, allowing proactive adjustments to maintain optimal performance in dynamic network conditions.

Figure 2 illustrates the computation offloading process where an initiator vehicle offloads all or part of a latency-sensitive task to edge servers via an access base station (BS), which coordinates with the network controller to determine the task allocation strategy [31]. The BS partitions the offloaded task into subtasks, distributes them across edge nodes for parallel processing, and compiles the results for the vehicle. Although task partitioning is complex, it is simplified here by assuming equal, independent subtasks [31].

IV. PROCEDURES FOR SYSTEM MODEL

The system model consists of a Mobile Edge Computing (MEC)-enabled vehicular network, which includes a base station (BS) with an MEC server and a mobile vehicle within the BS's coverage area. The vehicle has an on-chip microprocessor with limited computing power, while the MEC server has a high-performance processor. For tasks that require significant computation and are sensitive to delays, the vehicle offloads part of its workload to the MEC server to achieve faster processing. Figure 3 illustrates the structure of this system model. In Fig 3, the Greyscale, RGB (red/green/blue), and CMYK (cyan/magenta/yellow/black) are the vehicle moving down a highway; its distance from the base station (BS) alters with time [32]. This varying distance affects the communication between the vehicle and the BS, influencing the transmission rate and power gain.

The BS is positioned at the coordinates $(0, r, h)$, where r represents the distance from the BS to the highway, and h is the height of the BS antenna. The vehicle moves along the highway starting from the coordinates $(l, 0, 0)$ at a constant speed v . As in [33], The following equation can describe the time-dependent distance between the vehicle and the BS:

$$D_{(t)} = \sqrt{r^2 + h^2 + (l + vt)^2} \quad (1)$$

A. Two-Stage Computation Model

Computation offloading involves transferring delay-sensitive, computationally heavy tasks partially or fully from

TABLE I
DIFFERENT RESEARCH WORK, ALONG WITH THEIR KEY CONTRIBUTIONS AND LIMITATIONS

citation	Method and materials	Key elements	Contributions	Limitation
[21] Yuan & Su, 2023	Employs game-theoretic strategies for offloading in 5G MEC networks, validated via simulations.	Centralized management, combined with D2D and MEC offloading, and utility maximization.	-Presented centralized offloading. -Enhanced efficiency of resource management and system utility.	Lacking real-world implementation.
[22] Gao et al., 2023	Three-stage Stackelberg game model for edge-cloud collaboration, focusing on pricing and caching.	TEBI algorithm, joint optimization of offloading, caching, and pricing.	-Presented the TEBI algorithm for the game model. -Improved utility and reduced delays.	Vehicle Mobility issue.
[23] Zhou et al., 2022	Implements Stackelberg game-based optimization for computation offloading in cloud-edge setups.	Utility functions for cloud and edge servers, backward induction for strategy optimization.	-Presented the backward induction method in the gradient-based iterative search algorithm (GISA). -Improved task execution performance.	Neglected real-time network changes.
[24] R. Zhang et al., 2024	Online incentive mechanisms with privacy-preserving auction frameworks for UAV-assisted MEC.	Dynamic UAV scheduling, trajectory optimization, and privacy-preserving mechanisms.	-Discussed Privacy-preserving auction framework Ptero. -Reduced social cost.	Lacks a detailed exploration of general applications.
[25] Dai et al., 2023	Proposes a hybrid Stackelberg-auction game for task offloading in marine edge computing.	Two-tier task offloading, underwater acoustic and RF transmissions, and incentive mechanisms.	-Presented optimal task offloading strategies -Improves utility in marine environments.	Limited focus on real-world implementation.
[26] Qin et al., 2022	Employs Lyapunov optimization for energy-efficient task offloading in vehicular collaborative edge networks.	Task splitting, local/offloaded computation, UCB-based matching learning.	-Presented TSLRA, QOCA-UCB, and GECRA algorithms. -Reduces energy consumption under uncertainty and delay constraints.	Require to minimize the drift minus penalty in each time slot.
[27] P. Wang et al., 2023	Introduces a graph-optimized algorithm for urban tracking in ITS leveraging crowd-AI hybrid approaches.	Trajectory prediction, graph optimization, and task allocation.	-Discussed trajectory predicating algorithm -Reduces data offloading costs.	Lacks general applicability to other scenarios except urban tracking.
[9] Du & Fang, 2024	It combines the Generative Diffusion Model and the TD3 algorithm for MEC task offloading in Wi-Fi networks.	Generative AI, TD3, Hungarian algorithm for resource allocation.	-Discussed DTD3 Offloading Decision Algorithm -Reduced latency, energy consumption, and training costs.	Sparse evaluations, and limitations for varying network configurations.
[10] David et al., 2024	Uses Generative AI for traffic simulation, protocol optimization, and vehicular security.	Traffic optimization via GANs, adaptive communication, and security enhancement.	-Explore the potentiality of GAI -Improve traffic flow prediction and security defenses.	Challenges in scalability, real-time processing, and privacy integration.
[13] Xie et al., 2024	Proposes GAI-IoV framework for edge-based vehicular intelligence using collaborative inference.	Fine-tuning, distributed inference, and federated learning.	-Presented solutions via the SGHS algorithm and the backward induction method -Enhanced edge AI efficiency, reduced latency, and personalized content generation.	Privacy concerns and limited edge computational resources.
This Paper	Utilizes a Stackelberg game model to optimize computation offloading and resource allocation in MEC-enabled vehicular networks and uses Generative AI to find offloading costs.	Two-stage computation model, trade-off analysis of latency and energy, optimization via KKT conditions.	-Analyze Two-Stage Optimization and Generative AI-Based Offloading -Reduced computation overhead, enhanced energy efficiency, and scalability through dynamic resource allocation.	Assumes simplified network conditions; lacks comprehensive exploration of mobility impacts.

a vehicle to an MEC server, requiring vehicles to choose between local execution and offloading for efficiency [34]. The three parameters defining the task are L (task size in bits), C (computation intensity in CPU cycles per bit), and T_{max} (maximum allowable delay in seconds). Like the assumption in [35] and [36], The task can be split into two parts: $(1 - \alpha_{off})L$ bits for local computing and αL bits for MEC server computing, where α_{off} is the offloading ratio. The process includes:

1) *Local Computing*: The vehicle processes a portion of the task locally using its on-chip processor. This involves

calculating the local computing delay and energy consumption based on the local CPU frequency and the amount of data processed locally. For the portion $(1 - \alpha_{off})L$ that is processed locally by the VU, as in [32] the computation delay is given by:

$$T_l = \frac{(1 - \alpha_{off})LC}{f_l} \quad (2)$$

where f_l represents the local CPU frequency (in CPU cycles per second). As in [38], the energy consumption for this local processing is expressed as:

$$E_l = \zeta f_l^3 T_l = \zeta LC f_l^2 (1 - \alpha_{off}) \quad (3)$$

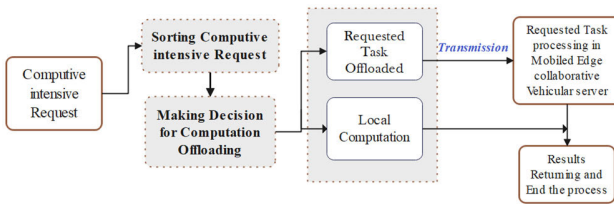


Fig. 1. Process of executing computation offloading.

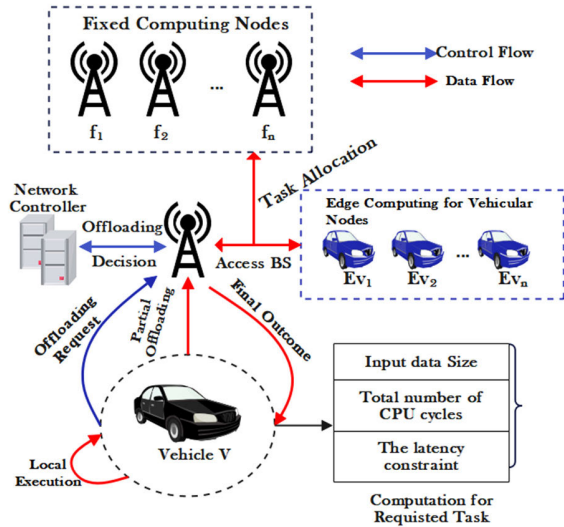


Fig. 2. The computation offloading in mobile edge collaborative vehicular networks.

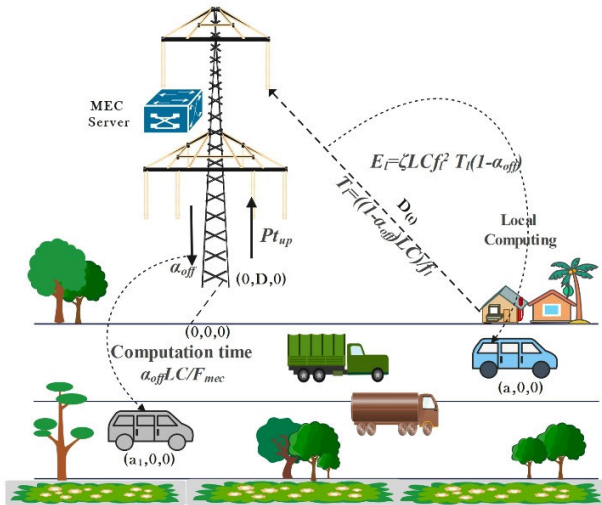


Fig. 3. The structure of the system model.

Here, ζ is a coefficient related to the energy consumption characteristics of the chip architecture.

2) *Computation Offloading*: The vehicle offloads a portion of the task to the MEC server. The offloading process includes uploading the data to the MEC server, processing the data on the server, and downloading the results back to the vehicle. The uploading time and the energy consumed during the upload is critical factors. When the portion $\alpha_{off}L$ is offloaded to the MEC server, the total processing delay T_r includes the time to upload the data, according to [32] the computation

time on the MEC server, and the time to download the results:

$$T_r = t_{up} + t_{comp} + t_{dn} \quad (4)$$

The computation time at the MEC server is: $t_{comp} = \frac{\alpha_{off}LC}{F_{mec}}$.

Where F_{mec} as the computation capability of the MEC server. The upload time t_{up} depends on the transmission rate, and the download time T_{dn} is typically negligible due to the small size of the returned data similar to [39], [40], and [41]. The energy consumption during offloading, primarily due to the upload process, is: $E_r = P t_{up}$; where P represents the transmit power of the VU. The goal is to use data and computing resources best, such as transmit power, uploading time, offloading ratio, and local CPU frequency. The aim is to reduce the overall computational overhead, which is the weighted sum of energy use and latency. The two-stage computation paradigm optimizes resource allocation by first determining the optimal offloading ratio based on task and network conditions, then allocating resources considering latency, energy, and efficiency. Unlike single-stage models, it supports flexible optimization and aligns with Stackelberg game structures for strategic resource management.

As both local computing and offloading tasks occur simultaneously, the total time for the User's Vehicles (UV) to complete its task is defined as:

$$T = \max(T_l, T_r) = \max\left(\frac{(1 - \alpha_{off})LC}{f_l}, t_{up} + \frac{\alpha_{off}LC}{F_{mec}}\right) \quad (5)$$

As in [32], The energy required for the UV to complete the task can be written as:

$$E = E_l + E_r = \zeta LC f_l^2 (1 - \alpha_{off}) + P t_{up} \quad (6)$$

B. Problem Formulation for Computation Offloading

For a given upload time (t_{up}), we need to optimize three variables: transmit power P , offloading ratio α_{off} , and latency of the system (T). The goal is minimizing total cost, balancing delay and energy, by optimizing task offloading from a user's vehicle (UV) to a MEC server. Using KKT conditions, the model optimizes offloading ratio, transmit power, and CPU frequency under constraints, efficiently balancing computational performance and energy consumption while ensuring UV stays within MEC range [32].

The restrictions set by the constraints guarantee that local CPU frequency, transmit power, and offloading ratio remain within their permitted range. We use mathematical techniques to find the optimal values of P , α_{off} and T . By following these steps, we can efficiently determine the best way to offload and compute tasks, minimizing the overall cost in terms of both time and energy, adopted from [32]. The major goal is to emphasize reducing the computation offloading brought on by both computation and communication. The offloading is prepared as a weighted combination of both latency and energy consumption, as in [32], denoted by $\beta_T T + \beta_E E$, where β_T and β_E are the respective weights

for latency and energy consumption. this optimization problem can be expressed as $\min_{\alpha_{off}, p, t_{up}, f_l} \beta_T T + \beta_E E$. Derived the Lagrangian formulation for λ_i (the Lagrange multipliers) from this equation as follows:

$$\mathcal{L}_{(p, \alpha_{off}, t_{up}, f_l, \lambda_i)} = \beta_T T + \beta_E E + \sum_{i=1}^8 \lambda_i g_i(p, \alpha_{off}, t_{up}, f_l) \quad (7)$$

Introducing each λ_i is associated with constraint equations g_i adopted from [32] and [33] as follows:

$$\alpha_{off} L + \varphi(p, t_{up}) = 0 \quad (8a)$$

$$R_{max} - \sqrt{r^2 + (l + vt)^2} = 0 \quad (8b)$$

$$0 \leq \alpha_{off} \leq 1 \quad (8c)$$

$$0 \leq p \leq p_{max} \quad (8d)$$

$$0 \leq f_l \leq F_{max} \quad (8e)$$

$$\lambda_i \geq 0, \forall i \in \{1, 2, \dots, 8\} \quad (8f)$$

Constraint equation (8a) decides the job size to be offloaded. The BS-VU connectivity is established below maximum transmission range through equation (8b). Constraints (8c), (8d), and (8e) ensure that the offloading rate, the transmission power, and the local CPU speed are not more than their respective limits, respectively. Equation (8f) is the Lagrange multiplier necessary to be non-negative individually. The Lagrangian for the constrained optimization problem is formed by introducing Lagrange multipliers $\lambda_1, \lambda_2, \dots, \lambda_8$ for these constraints:

$$\begin{aligned} \mathcal{L}(p, \alpha_{off}, t_{up}, f_l, \lambda_i) = & \beta_T T + \beta_E E + \lambda_1 (\alpha_{off} L + \varphi(p, t_{up})) \\ & + \lambda_2 \left(R_{max} - \sqrt{r^2 + (l + vt)^2} \right) \\ & + \lambda_3 (\alpha_{off} - 1) + \lambda_4 (-\alpha_{off}) \\ & + \lambda_5 (p - p_{max}) + \lambda_6 (-p) \\ & + \lambda_7 (f_l - F_{max}) + \lambda_8 (-f_l) \end{aligned} \quad (9)$$

The KKT conditions are a set of necessary conditions for a solution in nonlinear programming to be optimal. They include:

- Stationarity:** The gradient of the Lagrangian with respect to the decision variables $p, \alpha_{off}, t_{up}, f_l$ must be zero at the optimum. $\frac{\partial \mathcal{L}}{\partial p} = 0, \frac{\partial \mathcal{L}}{\partial \alpha_{off}} = 0, \frac{\partial \mathcal{L}}{\partial t_{up}} = 0, \frac{\partial \mathcal{L}}{\partial f_l} = 0$ Primal Feasibility: The constraints (8a)–(8f) must be satisfied.
- Dual Feasibility:** The Lagrange multipliers must be non-negative. $\lambda_1 \geq 0, \lambda_2 \geq 0, \dots, \lambda_8 \geq 0$
- Complementary Slackness:** The product of each Lagrange multiplier and its corresponding constraint must be zero, shown as below: $\lambda_3 (\alpha_{off} - 1) = 0, \lambda_4 (-\alpha_{off}) = 0, \lambda_5 (p - p_{max}) = 0, \lambda_6 (-p) = 0$

Applying stationary conditions to solve the optimal variables, we now take the partial derivatives of the Lagrangian function concerning the decision variables $p, \alpha_{off}, t_{up}, f_l$ and set them to zero.

1) Derivative With Respect to p : From the energy equation $E = \zeta LC f_l^2 (1 - \alpha_{off}) + P t_{up}$, we have, $\frac{\partial E}{\partial p} = t_{up}$. Additionally, from the constraint $\varphi(p, t_{up}) = \alpha_{off} L$, we take the derivative with respect to p : $\frac{\partial \varphi}{\partial p} = \frac{\partial (\alpha_{off} L)}{\partial p} = 0$. Thus, the equation becomes, $\beta_E t_{up} + \lambda_5 - \lambda_6 = 0$

2) Derivative With Respect to α_{off} : From the latency equation $T = \max(\frac{(1 - \alpha_{off}) LC}{f_l}, t_{up} + \frac{\alpha_{off} LC}{F_{mec}})$, we have two cases for T . To simplify, let's consider the partial derivative under both cases:

Case-1: $T = \frac{(1 - \alpha_{off}) LC}{f_l}$. Here, the partial derivative with respect to α_{off} is: $\frac{\partial T}{\partial \alpha_{off}} = -\frac{LC}{f_l}$; From the energy equation $E = \zeta LC f_l^2 (1 - \alpha_{off}) + P t_{up}$, we take the derivative with respect to α_{off} : $\frac{\partial E}{\partial \alpha_{off}} = -\zeta LC f_l^2$. Thus, the equation becomes: $-\beta_E \frac{LC}{f_l} - \beta_E \zeta LC f_l^2 + \lambda_1 L - \lambda_3 + \lambda_4 = 0$.

Case-2: $T = t_{up} + \frac{\alpha_{off} LC}{F_{mec}}$. Here, the partial derivative with respect to α_{off} is: $\frac{\partial T}{\partial \alpha_{off}} = \frac{LC}{F_{mec}}$. In this case, the equation becomes: $\beta_T \frac{LC}{F_{mec}} - \beta_E \zeta LC f_l^2 + \lambda_1 L - \lambda_3 + \lambda_4 = 0$.

3) Derivative Concerning t_{up} : From the latency equation, $T = t_{up} + \frac{\alpha_{off} LC}{F_{mec}}$, we have: $\frac{\partial T}{\partial t_{up}} = 1$. From energy equation $E = \zeta LC f_l^2 (1 - \alpha_{off}) + P t_{up}$, we take the derivative with respect to t_{up} and got $\frac{\partial E}{\partial t_{up}} = p$. Additionally, from the constraint $\varphi(p, t_{up}) = \alpha_{off} L$, we take the derivative with respect to t_{up} : $\frac{\partial \varphi}{\partial t_{up}} = \frac{\partial (\alpha_{off} L)}{\partial t_{up}} = 0$. Thus, the equation becomes: $\beta_T + \beta_E p = 0$.

4) Derivative Concerning f_l : From the latency equation, $T = \frac{(1 - \alpha_{off}) LC}{f_l}$, we have: $\frac{\partial T}{\partial f_l} = -\frac{(1 - \alpha_{off}) LC}{f_l^2}$. From energy equation $E = \zeta LC f_l^2 (1 - \alpha_{off}) + P t_{up}$, we take the derivative with respect to f_l : $\frac{\partial E}{\partial f_l} = 2\zeta LC f_l (1 - \alpha_{off})$. Thus, the equation becomes: $-\beta_T \frac{(1 - \alpha_{off}) LC}{f_l^2} + \beta_E 2\zeta LC f_l (1 - \alpha_{off}) + \lambda_7 - \lambda_8 = 0$.

Now these are the equations for the system that must be solved together with the constraints. To summarize, the system includes:

- $\beta_E t_{up} + \lambda_5 - \lambda_6 = 0$
- $\lambda_1 L - \lambda_3 + \lambda_4 - \beta_E \frac{LC}{f_l} - \beta_E \zeta LC f_l^2 = 0$
- $\beta_T + \beta_E p = 0$
- $\beta_E 2\zeta LC f_l (1 - \alpha_{off}) - \beta_T \frac{(1 - \alpha_{off}) LC}{f_l^2} + \lambda_7 - \lambda_8 = 0$

C. Two-Stage Computation Offloading Algorithm

The Two-Stage computation Offloading Algorithm (Algorithm 1) is designed to facilitate the process between Autonomous Vehicles (AVs) and a Roadside Base Station (BS). Based on the Ellipsoid approach, below is a thorough analysis of the algorithm:

Step 1: Set Lagrange multipliers. Find the best solution for the two-stage issue formulation using parameters of the offloading service.

Step 2: Under the present Lagrange multipliers and restrictions, compute the ideal values of $(P^* \alpha_{off}^* t_{up}^* f_l^*)$.

Step 3: Adjust Lagrange multipliers to minimise the goal using the ellipsoid approach.

Step 4: Continue Steps 2 and 3 until the solution converges, i.e., when the values meet all constraints within a pre-defined threshold.

Algorithm 1 Two-Stage Optimization and Generative AI-Based Offloading

Initialize:

Define system parameters: Task size (L), computation intensity (C), local CPU frequency (f_l), MEC server capacity (F_{mec}), and maximum transmission power (P_{max}).

Set weights for latency (β_T) and energy consumption (β_E).

Initialize Lagrange multipliers λ_i for constraints.

Initialize an empty set of vehicles.

Task Generation Using Generative AI:

Initialize Parameters: Number of vehicles, MEC/local computation capacities, latency constraints.

Generate Tasks Using AI:

Generate task parameters (L_v, C_v, D_v) for each vehicle $v \in V$

Assign latency_constraints[i]

For each vehicle, simulate task size, CPU cycles, and latency constraints using random distributions.

For each vehicle $v \in V$:

Compute Local Computation Time: $T_{Local} = \frac{(1-\alpha_{off})LC}{f_l}$

Compute Offloading Computation Time: $T_{offload} = t_{up} + \frac{\alpha LC}{F_{mec}}$

Total Latency Constraint: $T = \max(T_{Local}, T_{offload})$

Compute Energy Consumption: $E = \zeta LC f_l^2 (1 - \alpha_{off}) + P t_{up}$

Optimization Problem:

Minimize: $\beta_T T + \beta_E E$

Subject to Constraints:

$\varphi(p, t_{up}) = \alpha L, R_{max} - \sqrt{r^2 + (l + vt)^2} = 0,$
 $0 \leq \alpha_{off} \leq 1, 0 \leq p \leq P_{max}, 0 \leq f_l \leq F_{max}, \lambda_i \geq 0, \forall i \in \{1, 2, \dots, 8\}$

KKT Conditions for Optimality:

Solve for Optimal Values:

Extract optimal values: $p, \alpha_{off}, t_{up}, f_l, \lambda_i$. to solve the KKT conditions

Compute Lagrangian function:

$\mathcal{L}(p, \alpha_{off}, t_{up}, f_l, \lambda_i) = \beta_T T + \beta_E E + \sum_{i=1}^8 \lambda_i g_i(p, \alpha_{off}, t_{up}, f_l)$

Stationarity Conditions: $(\partial \mathcal{L})/(\partial p) = 0, (\partial \mathcal{L})/(\partial \alpha_{off}) = 0,$

$(\partial \mathcal{L})/(\partial t_{up}) = 0, (\partial \mathcal{L})/(\partial f_l) = 0$

Primal Feasibility: Satisfy all constraints.

Dual Feasibility: $\lambda_i \geq 0$

Complementary Slackness: $\lambda_i g_i = 0$

Formulate system equations:

$\beta_E t_{up} + \lambda_5 - \lambda_6 = 0$

$\lambda_1 L - \lambda_3 + \lambda_4 - \beta_E \frac{LC}{f_l} - \beta_E \zeta LC f_l^2 = 0$

$\beta_T + \beta_E P = 0$

$\beta_E 2 \zeta LC f_l (1 - \alpha_{off}) - \beta_T \frac{(1 - \alpha_{off})LC}{f_l^2} + \lambda_7 - \lambda_8 = 0$

Compute Costs for Each Vehicle:

def generative_ai_decision(task_size, cpu_cycles, channel_gain):

 Compute costs for each vehicle using a for loop

for i in range(len(vehicle_tasks)):

 task = vehicle_tasks[i]

 task_size = task["task_size"]

 cpu_cycles = task["cpu_cycles"]

Local Cost Calculation:

 local_latency = cpu_cycles/max_local_computation_capacity

 local_energy = 1e-27 * (max_local_computation_capacity ** 2) * cpu_cycles

 local_cost = 0.8 * local_latency + 0.2 * local_energy

Offloading Cost Calculation:

 upload_rate = (channel_gain * transmit_power)/noise_power

 upload_time = task_size/upload_rate

 offload_latency = upload_time + (cpu_cycles/ F_{mec})

 offload_energy = transmit_power * upload_time

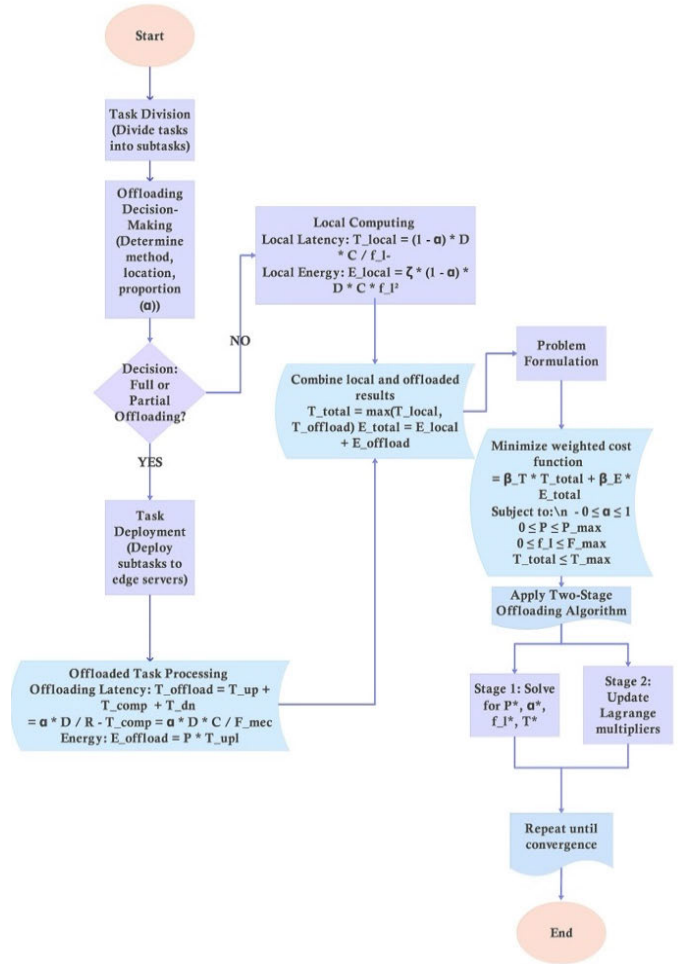


Fig. 4. Detailed flowchart mapping for methodology.

offload_cost = 0.8 * offload_latency + 0.2 * offload_energy

Compare Costs:

if local_cost < offloading_cost:

 total_cost = local_cost

else:

 total_cost = offloading_cost

return total_cost, local_cost, offloading_cost

Decision Making:

if $T_{offload} < T_{Local}$ and $T_{offload} < D_v$ then offload the task to MEC.

else: offload locally

Output:

Optimized values: Offloading ratio (α_{off}), Transmission power (P), Upload time (t_{up}), Local CPU frequency (f_l).

Finalize the offloading decision for each vehicle.

Print optimal cost for each vehicle

Fig. 4. Shows the detailed Flowchart Mapping for Methodology. This algorithm flowchart illustrates the decision-making and optimization process for computation offloading in MEC environments, highlighting task division, offloading decisions, using a two-stage algorithm until convergence. In a system where duties are offloaded from a User's Vehicle (UV) to a Mobile Edge Computing (MEC) server, the main equations in this optimisation issue describe the trade-offs between latency and energy usage. Considering both local computation

TABLE II
PARAMETER LIST

Parameters	Values	Description
t_{up}	0.1	Uploading time in seconds
f_l	1.0	Local CPU frequency in GHz
C	1900/8	Computation intensity in CPU cycles per bit
F_{max}	1.2	Maximum Local CPU frequency in GHz
R_{max}	100	Maximum Transmission range in meters
β_T	0.5	Weighted Sum of Latency
β_E	0.5	Weight for energy consumption
ζ	1.25×10^{-26}	Effective switched capacitance
F_{mec}	4.0	MEC server computation capability in GHz
L	1	Task input data size in MB
P	1.0	Transmit power in watts
α_{off}	0.5	Offloading ratio
T_{max}	5	Maximum allowable delay in seconds
P_{max}	23	Max Transmit power in dBm

and offloading, the latency equation reflects the entire time needed to finish an operation. Here, α_{off} is the portion of the task offloaded, L the task size, C the computational intensity, f_l the local CPU frequency, t_{up} the upload time, and F_{max} , the MEC server's computing capability. The energy equation considers the energy used in data transport and local computing. Reflecting the relative relevance of latency and energy, the optimisation problem minimises the weighted total of delay and energy. The limitations provide reasonable bounds on the offloading ratio, transmit power, and local CPU frequency, while also considering the distance between the UV and the MEC server. These equations and restrictions together simulate real-world situation in which effective task offloading balances computational speed and energy consumption, hence guaranteeing optimal performance within system constraints.

The above parameter list in Table II specifies important variables necessary for modelling and examining a computation offloading system in a Mobile Edge Computing (MEC) context. Energy use during local execution is affected by the computational intensity C in CPU cycles per bit mixed with ζ (effective switching capacitance). Guiding the system to strike a balance between performance and energy efficiency, the weights β_T and β_E denote the trade-off between latency and energy in the optimisation goals. The computation offloading process is governed by F_{mec} (MEC server computing capabilities) and p (transmit power), which therefore define the viability and efficiency of offloading workloads to the MEC server. The offloading ratio α_{off} specifies the share of tasks sent to the MEC server, hence allowing the splitting of workloads between local and offloaded execution. Aiming for the least delay, lower energy use, and effective job execution, these variables taken together offer a thorough basis for creating and fine-tuning compute offloading techniques.

Reflecting real-world choices like urban traffic congestion, highway driving, and rural road conditions, the offloading ratio

α_{off} defines how much of a computational task is processed locally versus being sent to a Mobile Edge Computing (MEC) server, therefore improving the validation for the framework on resource allocation. These situations would permit a realistic evaluation of the performance of the system under different mobility patterns, communication settings, and work demands. The transmit power directly affects battery life in devices like smartphones or autonomous cars, since it is the energy needed for wireless communication. The local CPU frequency and MEC server capacity model the processing rates of the local device and the server, respectively, hence affecting how fast jobs are finished. The upload time and task size L relate to network conditions and the complexity of the task affecting signal strength and latency. The weights β_T and β_E allow system designers to prioritize either faster task completion or energy efficiency, depending on application requirements. Together, these parameters model real-world trade-offs in mobile computing, where optimizing latency and energy consumption is crucial for performance and user experience.

V. RESULTS AND CONTRIBUTION

Using Python for the numerical solution of this system to calculate the optimal values for $P, \alpha_{off}, t_{up}, f_l$. The numerical solution for the optimization problem provided the following graphical representation in Figure 5.

The graph in fig. 5(a) highlights the trade-off between latency and energy consumption as the latency weight (β_T) varies, showing that prioritizing low latency increases energy usage. This insight helps optimize collaborative vehicular networks by adjusting (β_T) based on application needs. The trends for different system parameters validate a game-theoretic approach for dynamic resource allocation. The computation model adapts offloading strategies based on task size, offloading larger tasks to edge servers to reduce local computation time while processing smaller tasks locally to minimize communication overhead. Fig. 5(b) shows that the computation model achieves the lowest computation overhead, especially for high-intensity tasks, demonstrating superior efficiency and scalability. Practical applications in autonomous driving, augmented reality (AR), and industrial IoT highlight the trade-off between latency and computation overhead, emphasizing the need for efficient offloading strategies for higher system performance, where real-time processing is essential.

The two-level solution approach performs moderately well but lags the proposed method, while the SDR-based method exhibits significantly higher overhead, especially as task computation intensity increases.

This comparison highlights the effectiveness of the model in reducing computational overhead, making it highly suitable for resource-constrained and high-demand scenarios in mobile edge collaborative vehicular networks. The findings validate the importance of advanced optimization strategies to ensure energy-efficient and computation-effective resource management.

Figure 5(c) shows that increasing maximum transmit power reduces computation overhead by accelerating data

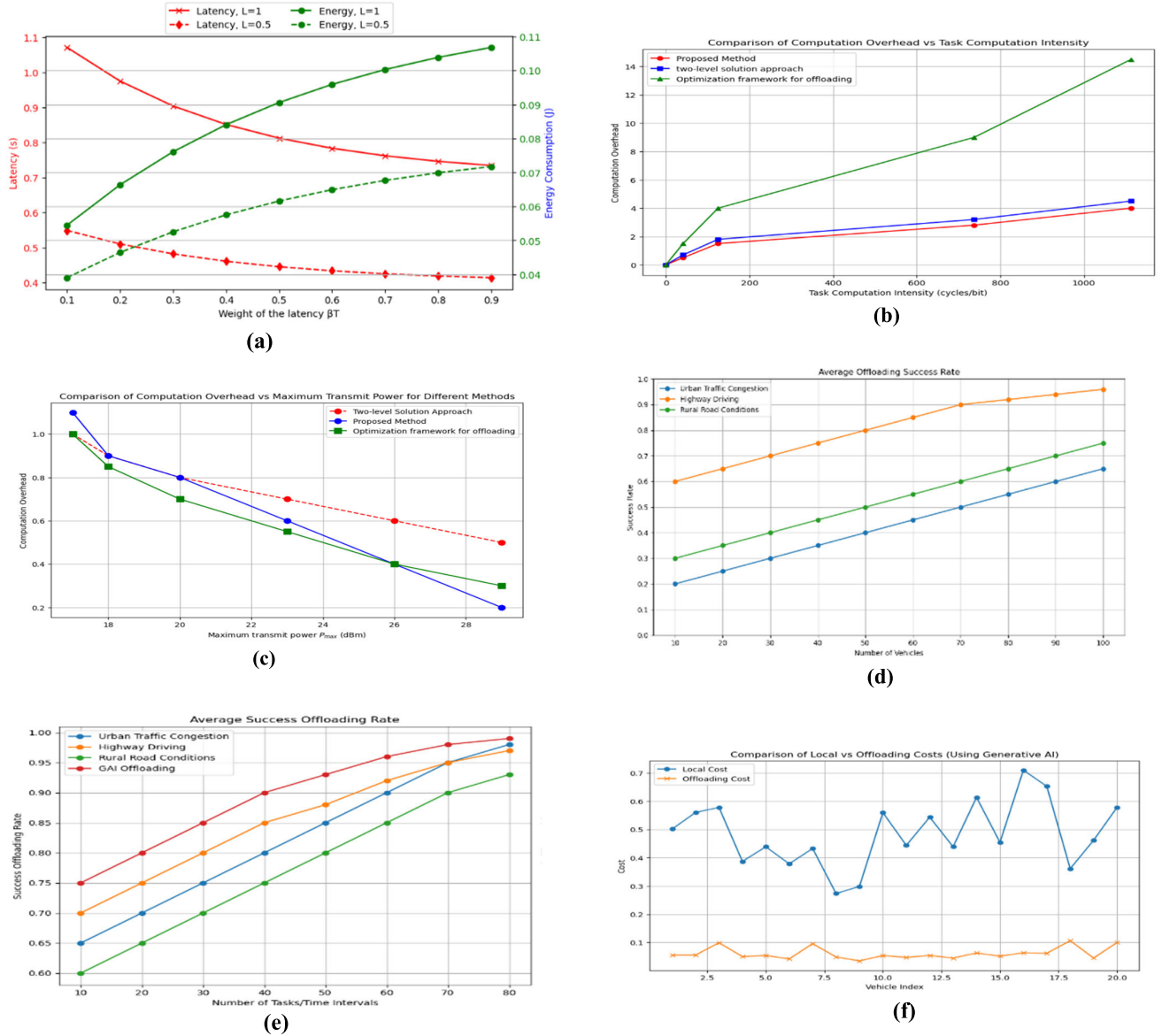


Fig. 5. (a). Performance vs weight of latency β_T . (b). Computation overhead vs task computation intensity C . (c). Computation overhead vs maximum transmit power. (d). Average offloading success rate in vehicular mobility under different scenarios. (e). Average offloading success rate under different scenarios for different number of tasks/time intervals. (f). Comparison of local vs offloading costs using generative AI.

transmission and lowering time costs. By dynamically adjusting transmission power and offloading ratios, this GAI-Game theory model enhances resource utilization, latency reduction, and energy efficiency, providing valuable insights for optimizing communication strategies in vehicular and mobile edge networks. Fig. 5(d) and Fig. 5(e) illustrate the Average Offloading Success Rate across different vehicular mobility scenarios—Urban Traffic Congestion, Highway Driving, and Rural Road Conditions. Urban areas achieve the highest success rate due to frequent stops and stable connectivity, whereas highway driving maintains a steady rate, and rural roads show the lowest success rate due to sparse connectivity and limited access to edge servers.

The integration of Generative AI (GAI) offloading further improves offloading efficiency, particularly in complex and resource-constrained environments. These results underline the need to customise offloading techniques to various mobility

situations for improved system performance. By means of traffic management, using V2X communication, and using eco-driving policies, this study aims to reduce energy use during high vehicle mobility. Results indicate that in dynamic traffic conditions, predictive analytics and machine learning models increase efficiency by 10-12%, while smart routing algorithms can cut energy use by up to 15-20%. Including renewable energy sources and optimising infrastructure further increases energy savings; autonomous car fleets use 25-30% less energy than conventional systems. From rural locations with low resources to urban areas with high-density edge servers, this flexibility guarantees that the suggested architecture can run efficiently in various MEC settings.

Using a Generative AI model, Fig. 5(f) contrasts local processing with offloading costs. The findings show that offloading workloads to Mobile Edge Computing (MEC) servers consistently lowers computation costs when compared

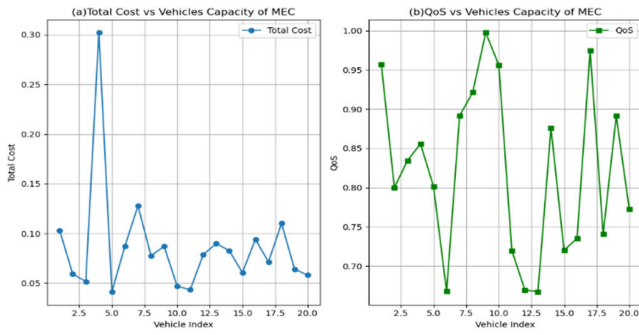


Fig. 6. The performance of algorithms with varying capacities of MEC: (a) total cost vs. Vehicles capacity of MEC, (b) QoS vs. Vehicles capacity of MEC.

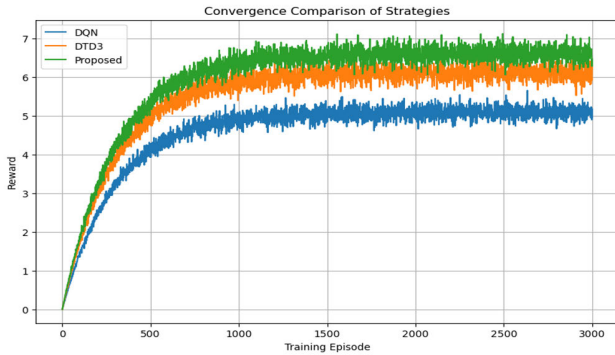


Fig. 7. Convergence comparison between DTD3 and proposed work.

to local processing. Although local prices range among vehicles due to CPU capacity and task complexity, offloading guarantees consistent and lower costs, proving its efficiency. By predicting compute needs and latency limits, generative artificial intelligence allows dynamic changes in offloading ratios and resource allocation for effective task execution. By balancing latency and energy use, the Stackelberg game model lets the system fit various tasks, from basic data processing to computationally demanding applications.

Two graphs—total cost vs. vehicle capacity of MEC and Quality of Service (QoS) vs. vehicle capacity of MEC—allow Figure 6 to investigate algorithm performance under different MEC capacities. Higher MEC capacity generally reduces computation costs, although occasional spikes due to resource bottlenecks or inefficient task allocations indicate dynamic network challenges, resolved by Generative AI-driven offloading. The second graph demonstrates improved QoS with increasing MEC capacity, despite fluctuations highlighting workload and mobility heterogeneity; the proposed Generative AI framework effectively predicts task parameters, enhancing resource allocation and ensuring consistently high QoS.

By means of cumulative rewards over training episodes, Figure 7 contrasts the convergence performance of three strategies—DQN, DTD3, and the Proposed Method. Indicating better computing efficiency and resource management, the suggested approach converges faster and obtains more cumulative rewards than DTD3 and DQN. Generative AI's predictive abilities, which dynamically optimise offloading choices instead of depending on static heuristic techniques, account for this benefit. Unlike conventional methods, Gener-

ative AI can simulate many situations in real-time, changing resource allocation for best performance.

All things considered, next-generation intelligent transportation systems (ITS) depend on these results since they support augmented reality, real-time traffic monitoring, and autonomous driving. Future research will be strongly based on the understanding of trade-offs between computational overhead, delay, and energy use.

Developing more efficient resource management solutions for automotive networks, IoT applications, and edge computing will be greatly aided by optimizing offloading mechanisms, using Generative AI and game-theoretic models. The findings underline the need of adaptive and scalable techniques to improve efficiency, lower energy usage, and preserve good QoS in actual deployments.

VI. CONCLUSION

In order to optimise the trade-off between latency and energy consumption, this paper analyzes a state-of-the-art game-theoretic framework for managing resources and offloading computing in mobile edge collaborative vehicle networks. Its flexibility makes it very relevant to autonomous vehicle networks and intelligent transportation systems, where major issues are changing environments and resource limits. The computation model assures energy-efficient and latency-aware operations by combining dynamic resource allocation with compute offloading, hence preserving scalability and resilience. It improves Quality of Service (QoS) in vehicle edge networks, providing a basic answer for handling computational issues in mobile edge computing settings needing effective real-time resource control.

But, using the game-theoretic and Generative AI (GAI) framework raises issues in security, privacy, and scalability. While privacy issues result from sharing sensitive job and location data, insecure communication channels create security concerns. Although the finding offers insightful analysis of the performance of the GAI-Game theory model, it has some drawbacks in accurately representing real-world settings. For instance, the simulations may not fully capture the complexity of vehicular mobility patterns and dynamic network conditions. Scalability remains a challenge, especially in dense urban settings. Future research should explore encryption, federated learning, adaptive resource allocation, real-time detection, and dynamic task prioritization to enhance robustness, efficiency, and real-world applicability.

ACKNOWLEDGMENT

This work was conducted and funded by the National University of Malaysia under the Research Grant Scheme No. GUP 2023-010. The authors are thankful to the Dean-ship of Graduate Studies and Scientific Research at Najran University for partial funding of this work under the Growth Funding Program grant code (NU/GP/SERC/13/358-1). This work was conducted in collaboration with the Network and Communication Technology Lab, Faculty of Information Science and Technology, The National University of Malaysia, and the Applied AI Research Laboratory at Texas A&M University—Victoria, Texas.

REFERENCES

- [1] W. Lv et al., "Energy consumption and QoS-aware co-offloading for vehicular edge computing," *IEEE Internet Things J.*, vol. 10, no. 6, pp. 5214–5225, Mar. 2023, doi: [10.1109/JIOT.2023.3221966](#).
- [2] C.-L. Chen, C. G. Brinton, and V. Aggarwal, "Latency minimization for mobile edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 22, no. 4, pp. 2233–2247, Apr. 2023, doi: [10.1109/TMC.2021.3117511](#).
- [3] R. Li, G. Chen, D. Gan, H. Gu, and J. Lü, "Stackelberg and nash equilibrium computation in non-convex leader-follower network aggregative games," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 71, no. 2, pp. 898–909, Feb. 2024, doi: [10.1109/TCSI.2023.3339753](#).
- [4] B. Yang, D. Wu, H. Wang, Y. Gao, and R. Wang, "Two-layer Stackelberg game-based offloading strategy for mobile edge computing enhanced FiWi access networks," *IEEE Trans. Green Commun. Netw.*, vol. 5, no. 1, pp. 457–470, Mar. 2021, doi: [10.1109/TGCN.2020.3044566](#).
- [5] S. Sharif, S. Zeadally, and W. Ejaz, "Resource optimization in UAV-assisted IoT networks: The role of generative AI," *IEEE Internet Things Mag.*, vol. 8, no. 1, pp. 34–41, Jan. 2025, doi: [10.1109/IOTM.001.2400045](#).
- [6] F. Morsidi and H. Budiman, "Influence of shortest route approximation on relegating urban area's transportation network priorities," *Asia-Pacific J. Inf. Technol. Multimedia*, vol. 12, no. 2, pp. 240–257, Dec. 2023.
- [7] R. Hartono, "Traffic cones detection for autonomous ground vehicle," *Asia-Pacific J. Inf. Technol. Multimedia*, vol. 11, no. 1, pp. 65–77, Jun. 2022, doi: [10.17576/apjtm-2022-1101-06](#).
- [8] R. Zhang et al., "Generative AI-enabled vehicular networks: Fundamentals, framework, and case study," *IEEE Netw.*, vol. 38, no. 4, pp. 259–267, Jul. 2024, doi: [10.1109/MNET.2024.3391767](#).
- [9] X. Du and X. Fang, "An integrated communication and computing scheme for Wi-Fi networks based on generative AI and reinforcement learning," 2024, *arXiv:2404.13598*.
- [10] T. David, K. Muhammad, K. Nassisi, and B. Farus, "Enhancing vehicular networks with generative AI: Opportunities and challenges," 2024, *arXiv:2407.11020*.
- [11] L. Yang, A. Moubayed, and A. Shami, "MTH-IDS: A multitiered hybrid intrusion detection system for Internet of Vehicles," *IEEE Internet Things J.*, vol. 9, no. 1, pp. 616–632, Jan. 2022, doi: [10.1109/JIOT.2021.3084796](#).
- [12] A. S. Zaini, S. Norul, H. Sheikh, A. Abdullah, N. B. Sana, and S. Basiron, "Exploitable edge analysis for free flow vehicle plate localization," *Asia-Pacific J. Inf. Technol. Multimedia*, vol. 10, no. 1, pp. 27–42, 2021.
- [13] G. Xie et al., "GAI-IoV: Bridging generative AI and vehicular networks for ubiquitous edge intelligence," *IEEE Trans. Wireless Commun.*, vol. 23, no. 1, pp. 12799–12814, Oct. 2024, doi: [10.1109/TWC.2024.3396276](#).
- [14] B. Cao, Z. Li, X. Liu, Z. Lv, and H. He, "Mobility-aware multiobjective task offloading for vehicular edge computing in digital twin environment," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 10, pp. 3046–3055, Oct. 2023, doi: [10.1109/JSAC.2023.3310100](#).
- [15] J. Y. Wu, R. Xin, J. B. Zhao, T. Zheng, D. Jiang, and P. F. Zhang, "Study on delay optimization of fog computing edge nodes based on the CPSO-LB algorithm," *Wireless Commun. Mobile Comput.*, vol. 2020, no. 2, 2020, Art. no. 8811175, doi: [10.1155/2020/8811175](#).
- [16] M. K. Hasan et al., "Federated learning for computational offloading and resource management of vehicular edge computing in 6G-V2X network," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 3827–3847, Feb. 2024, doi: [10.1109/TCE.2024.3357530](#).
- [17] S. Chen et al., "Vehicle-to-everything (V2X) services supported by LTE-based systems and 5G," *IEEE Commun. Standards Mag.*, vol. 1, no. 2, pp. 70–76, Jun. 2017.
- [18] Y. Zhang, J. Zhang, S. Yue, W. Lu, J. Ren, and X. S. Shen, "Mobile generative AI: Opportunities and challenges," *IEEE Wireless Commun.*, vol. 31, no. 4, pp. 58–64, Aug. 2024, doi: [10.1109/MWC.006.2300576](#).
- [19] W. Zeng et al., "Generative AI-aided multimodal parallel offloading for AIGC metaverse service in IoT networks," *IEEE Internet Things J.*, vol. 12, no. 10, pp. 13273–13285, May 2025, doi: [10.1109/JIOT.2025.3535623](#).
- [20] C. He, W. Jiang, X. Wang, W. Wang, and X. Xie, "Generative AI-enhanced task offloading strategy for the IoV: An RSU-RSU load-balancing perspective," *IEEE Netw. Lett.*, 2025, doi: [10.1109/LNET.2025.3542094](#).
- [21] Y. Yuan and W. Su, "A game theory-based strategy for allocating and offloading computing resources in 5G networks," in *Proc. Int. Conf. Netw. Netw. Appl. (NaNA)*, Aug. 2023, pp. 92–97, doi: [10.1109/nana60121.2023.00023](#).
- [22] L. Gao, X. Chen, B. Yin, L. Cui, and H. Xing, "Game theory based task offloading, content caching and resource pricing under edge-cloud collaboration in 6G network," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, Mar. 2023, pp. 1–6, doi: [10.1109/WCNC55385.2023.10118802](#).
- [23] H. Zhou, Z. Wang, N. Cheng, D. Zeng, and P. Fan, "Stackelberg-game-based computation offloading method in cloud-edge computing networks," *IEEE Internet Things J.*, vol. 9, no. 17, pp. 16510–16520, Sep. 2022, doi: [10.1109/JIOT.2022.3153089](#).
- [24] R. Zhang, R. Zhou, Y. Wang, H. Tan, and K. He, "Incentive mechanisms for online task offloading with privacy-preserving in UAV-assisted mobile edge computing," *IEEE/ACM Trans. Netw.*, vol. 32, no. 3, pp. 2646–2661, Jun. 2024, doi: [10.1109/TNET.2024.3364141](#).
- [25] M. Dai, Z. Luo, Y. Wu, L. Qian, B. Lin, and Z. Su, "Incentive oriented two-tier task offloading scheme in marine edge computing networks: A hybrid Stackelberg-auction game approach," *IEEE Trans. Wireless Commun.*, vol. 22, no. 12, pp. 8603–8619, Dec. 2023, doi: [10.1109/TWC.2023.3264607](#).
- [26] P. Qin, Y. Fu, G. Tang, X. Zhao, and S. Geng, "Learning based energy efficient task offloading for vehicular collaborative edge computing," *IEEE Trans. Veh. Technol.*, vol. 71, no. 8, pp. 8398–8413, Aug. 2022, doi: [10.1109/TVT.2022.3171344](#).
- [27] P. Wang et al., "Graph optimized data offloading for crowd-AI hybrid urban tracking in intelligent transportation systems," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 1, pp. 1075–1087, Jan. 2023, doi: [10.1109/TITS.2022.3141885](#).
- [28] T. Zhang and W. Chen, "Computation offloading in energy harvesting aided heterogeneous mobile edge computing," in *Proc. IEEE 93rd Veh. Technol. Conf. (VTC-Spring)*, Apr. 2021, pp. 1–5, doi: [10.1109/VTC2021-Spring51267.2021.9448854](#).
- [29] Y. Sahni, J. Cao, L. Yang, and Y. Ji, "Multi-hop multi-task partial computation offloading in collaborative edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 5, pp. 1133–1145, May 2021, doi: [10.1109/TPDS.2020.3042224](#).
- [30] L. Li, T. Q. S. Quek, J. Ren, H. H. Yang, Z. Chen, and Y. Zhang, "An incentive-aware job offloading control framework for multi-access edge computing," *IEEE Trans. Mobile Comput.*, vol. 20, no. 1, pp. 63–75, Jan. 2021, doi: [10.1109/TMC.2019.2941934](#).
- [31] X. Hou et al., "Reliable computation offloading for edge-computing-enabled software-defined IoV," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 7097–7111, Aug. 2020, doi: [10.1109/JIOT.2020.2982292](#).
- [32] J. Wang, D. Feng, S. Zhang, J. Tang, and T. Q. S. Quek, "Computation offloading for mobile edge computing enabled vehicular networks," *IEEE Access*, vol. 7, pp. 62624–62632, 2019, doi: [10.1109/ACCESS.2019.2915959](#).
- [33] K. Zhang, S. Leng, X. Peng, L. Pan, S. Maharjan, and Y. Zhang, "Artificial intelligence inspired transmission scheduling in cognitive vehicular communications and networks," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 1987–1997, Apr. 2019, doi: [10.1109/JIOT.2018.2872013](#).
- [34] S. Melendez and M. P. McGarry, "Computation offloading decisions for reducing completion time," in *Proc. 14th IEEE Annu. Commun. Netw. Conf. (CCNC)*, Jan. 2017, pp. 160–164, doi: [10.1109/CCNC.2017.7983099](#).
- [35] Y. Wang, M. Sheng, X. Wang, L. Wang, and J. Li, "Mobile-edge computing: Partial computation offloading using dynamic voltage scaling," *IEEE Trans. Commun.*, vol. 64, no. 10, pp. 4268–4282, Oct. 2016, doi: [10.1109/TCOMM.2016.2599530](#).
- [36] Y. Dai, D. Xu, S. Maharjan, and Y. Zhang, "Joint load balancing and offloading in vehicular edge computing and networks," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4377–4387, Jun. 2019, doi: [10.1109/JIOT.2018.2876298](#).
- [37] J. Ren, G. Yu, Y. Cai, and Y. He, "Latency optimization for resource allocation in mobile-edge computation offloading," *IEEE Trans. Wireless Commun.*, vol. 17, no. 8, pp. 5506–5519, Aug. 2018, doi: [10.1109/TWC.2018.2845360](#).
- [38] W. Zhang, Y. Wen, K. Guan, D. Kilper, H. Luo, and D. O. Wu, "Energy-optimal mobile cloud computing under stochastic wireless channel," *IEEE Trans. Wireless Commun.*, vol. 12, no. 9, pp. 4569–4581, Sep. 2013, doi: [10.1109/TWC.2013.072513.121842](#).
- [39] F. Wang, J. Xu, X. Wang, and S. Cui, "Joint offloading and computing optimization in wireless powered mobile-edge computing systems," *IEEE Trans. Wireless Commun.*, vol. 17, no. 3, pp. 1784–1797, Mar. 2018, doi: [10.1109/TWC.2017.2785305](#).
- [40] S. Bi and Y. J. Zhang, "Computation rate maximization for wireless powered mobile-edge computing with binary computation offloading," *IEEE Trans. Wireless Commun.*, vol. 17, no. 6, pp. 4177–4190, Jun. 2018, doi: [10.1109/TWC.2018.2821664](#).

- [41] C. You, K. Huang, and H. Chae, "Energy efficient mobile cloud computing powered by wireless energy transfer," *IEEE J. Sel. Areas Commun.*, vol. 34, no. 5, pp. 1757–1771, May 2016, doi: [10.1109/JSAC.2016.2545382](https://doi.org/10.1109/JSAC.2016.2545382).



Nusrat Jahan received the B.Sc. and M.Sc. degrees (Hons.) in electrical and electronic engineering from Islamic University, Kushtia, Bangladesh. She is currently pursuing the Ph.D. degree (as a Postgraduate Student) with the Faculty of Information Science and Technology, Universiti Kebangsaan Malaysia.

Her research interests lie in digital speech processing, networking, database management systems, smart vehicular networks, and artificial intelligence.



Mohammad Kamrul Hasan (Senior Member, IEEE) received the Ph.D. degree in electrical and communication engineering from the Faculty of Engineering, International Islamic University, Malaysia, in 2016. He is an Associate Professor with the Faculty of Information Science and Technology, Center for Cyber Security, Universiti Kebangsaan Malaysia (UKM). He specializes in elements of cutting-edge information-centric networks, computer networks, data communication and security, mobile networks and privacy protection,

cyber-physical systems, industrial IoT, transparent AI, and electric vehicle networks. He has published over 150 indexed papers in ranked journals and conference proceedings. He is a member of the Institution of Engineering and Technology and the Internet Society. He served on the IEEE Student Branch as the Chair from 2014 to 2016. He has actively participated in many events/workshops/trainings for Malaysia's IEEE and IEEE Humanity programs. He is the general chair, the co-chair, and a speaker of conferences and workshops for the sake of society and academic knowledge building, sharing, and learning. He has been contributing and working as a volunteer for underprivileged people for the welfare of society. He is an Editorial Board Member of many prestigious high-impact journals, such as IEEE, IET, Elsevier, Frontier, and MDPI. He is a certified Professional Technologist in Malaysia.



Shayla Islam (Senior Member, IEEE) is the Deputy Director and the Head of Research with the Institute of Computer Science and Digital Innovation (ICS DI), UCSI University, Malaysia. She was selected as one of the Top 2% of scientists worldwide by Stanford University, USA, and Elsevier BV. Moreover, she has published more than 100 WoS/Scopus-indexed articles with high-impact factors, and ten of these articles have been in the top ten journals in the area. Currently, her H-index is 27, with 2,585 citations in Scopus. She

has also evaluated the Ph.D. thesis at the international level as the foreign examiner. Her research interests include mobile networks in a 5G environment, telecommunications, cyber-physical systems, artificial intelligence, and network security. She was awarded a Professional Technologist by the Malaysian Board of Technologists (MBoT). She is a Graduate Engineer of the Board of Engineers Malaysia and the Institution of Engineers Bangladesh (MIEB-M/40624). She is a Guest Editor and a reviewer of international, prestigious, and high-impact factor journals, such as *Expert Systems with Applications* and *Frontiers in Public Health*.



Mohd Zakree Ahmad Nazri received the Ph.D. degree from the Universiti Teknologi Malaysia (UTM), in 2011, specializing in metaheuristic algorithms for ontology learning. He is an Associate Professor and the Deputy Executive Director (Strategic Data) with the Strategic Centre of Universiti Kebangsaan Malaysia (UKM), where he leads initiatives in data-driven decision-making. With expertise spanning machine learning, decision support systems, and misinformation detection, he has focused on interdisciplinary research at the intersection of computational optimization and applied artificial intelligence. He co-founded and directed UKM's Master's Program in Data Science under the Centre for Artificial Intelligence Technology (CAIT). As a Principal Researcher with the CAIT's Data Mining and Optimization (DMO) Research Group, he focuses on developing decision support tools for complex real-world challenges. His work integrates optimization algorithms, social media analytics, and machine learning to enhance governance, resource allocation, and misinformation mitigation. He has authored over 70 peer-reviewed publications and serves on program committees for international conferences.



Khairul Akram Zainol Ariffin received the bachelor's and master's degrees (Hons.) in system engineering with computer engineering from the University of Warwick, U.K., in 2008 and 2009, respectively, and the Ph.D. degrees in information systems from the Universiti Teknologi PETRONAS (UTP). Then, he was appointed as a Researcher with the Digital Forensic Department, CyberSecurity Malaysia, and was entrusted with the research on embedded systems and live forensics. Currently, he is a member of the Center for Cyber Security, Faculty of Information Science and Technology, Universiti Kebangsaan Malaysia, to pursue his passion in research toward cybersecurity, digital forensics, algorithms, and embedded systems. He has published many journal articles and conference papers and has been published internationally. He is also a GCFA-certified.



Huda Saleh Abbas received the Ph.D. degree in network engineering from the Royal Melbourne Institute of Technology, Melbourne, Australia, in 2017. She is an Assistant Professor with the Department of Computer Science, College of Computer Science and Engineering, Taibah University, Madinah, Saudi Arabia. Her research interests include computer networks, wireless networks, optical networks, and edge computing.



Ali Alqahtani received the Ph.D. degree in computer engineering from Oakland University, Rochester Hills City, MI, USA, in 2020. He is currently an Assistant Professor with Najran University (NU). His research interests include machine learning in general and deep learning in image and signal processing, wireless vehicular networks (VANETs), wireless sensor networks, and cyber-physical systems.



Hardik Gohel is an Associate Professor and the Director of the Applied AI Research Laboratory, Texas A&M University–Victoria, Texas, USA. He is a principal investigator (PI) on federally funded research projects totaling over \$1 million, advancing impactful innovations in applied AI. His expertise lies in multidisciplinary applied artificial intelligence, with a focus on cybersecurity, digital healthcare, environmental remediation, and sustainable infrastructure.